

Reimplementação heterogênea (GPU + CPU multicore) do pipeline mbkmeans para scRNA-seq: onde a GPU compensa e o que o speedup revela sobre os dados

Christian de Souza Ramos¹, Giulio Henrique Borges Basso¹,
Igor Garbin Manzan Mazo¹, Wallisson Policarpo Teodoro¹

¹Bacharelado em Inteligência Artificial – Instituto de Informática
Universidade Federal de Goiás (UFG) – Goiânia, GO – Brasil

{christian.ramos@discente.ufg.br, giulio_henrique@discente.ufg.br,
igor_mazo@discente.ufg.br, wallisson.policarpo@discente.ufg.br}@discente.ufg.br

Abstract. We reimplement, on GPU (CuPy/CUDA) and multicore CPU (Numba/OpenMP), the single-cell RNA-seq analysis pipeline of Hicks et al. (2021), originally run on single-thread CPU, over a sparse matrix of $\sim 1.3\text{M}$ cells $\times$ $\sim 28\text{k}$ genes stored in HDF5. Beyond seeking acceleration, we study when the GPU actually pays off, showing by measurement that per-stage speedups range from $\sim 1.7\times$ (I/O-bound stages) to $1080\times$ (PCA), with an overall $\sim 232\times$ on the reimplemented pipeline. We reformulate PCA from an out-of-core iterative SVD into a two-pass gene $\times$ gene covariance eigendecomposition, a gain that is mostly algorithmic. Finally, using the silhouette coefficient — a metric absent from the reference paper — we provide evidence that the data are not well separated by K-means.

Resumo. Reimplementamos em GPU (CuPy/CUDA) e CPU multicore (Numba/OpenMP) o pipeline de análise de scRNA-seq de Hicks et al. (2021), executado originalmente em CPU single-thread, sobre uma matriz esparsa de $\sim 1,3$ milhão de células $\times$ ~ 28 mil genes armazenada em HDF5. Além de buscar aceleração, investigamos quando a GPU de fato compensa, mostrando por medição que os ganhos por etapa variam de $\sim 1,7\times$ (etapas limitadas por I/O) a $1080\times$ (PCA), com speedup global de $\sim 232\times$ no pipeline reimplementado. Reformulamos o PCA (de SVD iterativo out-of-core para covariância gene $\times$ gene em duas passadas), com ganho majoritariamente algorítmico. Por fim, usando a métrica silhouette, ausente no paper, evidenciando que os dados não são bem separados por K-means.

1. Introdução

A análise de dados de sequenciamento de RNA de célula única (scRNA-seq) é hoje uma das principais ferramentas para identificar subpopulações celulares a partir da expressão gênica. Um passo central dessa análise é o agrupamento (clustering) não supervisionado das células. Com o avanço das tecnologias, os conjuntos de dados cresceram de milhares para milhões de células, o que torna algoritmos clássicos como o K-means custosos ou inviáveis quando exigem carregar toda a matriz em memória [1].

Para tratar esse problema, [1] desenvolveram o pacote mbkmeans (R/Bioconductor), uma implementação on-disk do K-means de mini-lotes (mini-batch) que opera sobre representações em disco no formato HDF5, sem carregar todos os

dados de uma vez. Como demonstração, os autores processam um conjunto de $\sim 1,3$ milhão de células de cérebro de camundongo. O pipeline completo, executado pelos autores, foi estruturado em pré-processamento, *clusterização inicial*, normalização, redução de dimensionalidade (IRLBA-PCA) e agrupamento, em CPU single-thread, totalizando cerca de 104 horas, dominadas pelo IRLBA-PCA (96 h) e pela normalização (5,18 h).

Neste trabalho, reimplementamos esse pipeline em um ambiente heterogêneo, combinando GPU (CuPy/CUDA) para os trechos limitados por cálculo e CPU multicore (Numba/OpenMP) para os trechos limitados por movimentação de memória. Mais do que perseguir aceleração, tratamos a pergunta “a GPU é sempre mais rápida?” como objeto de estudo: medimos, etapa a etapa, onde a GPU compensa, onde a CPU é a escolha correta e por quê. Essa análise de trade-off de arquitetura não é feita no paper original, que executa tudo em CPU. Além disso, incorporamos uma métrica de qualidade de agrupamento (*silhouette score*) ausente no *paper*, que nos levou a uma observação inesperada sobre a natureza dos dados.

2. Formulação do problema

O dado de entrada é uma matriz esparsa X , tendo $\approx 55\text{GB}$, com $n \approx 1,3 \times 10^6$ células e $g \approx 2,8 \times 10^4$ genes, armazenada em HDF5 no formato CSR, com cerca de 2,4 bilhões de elementos não nulos. Sobre essa matriz, o pipeline executa cinco etapas encadeadas:

1. **Pré-processamento (QC/filtragem)**: equivalente ao *scater* do *paper*: calcula métricas de controle de qualidade e descarta células/genes.
2. **Clustering inicial (mini-batch K-means)**: passo intermediário que serve de entrada para a normalização (não é o resultado final).
3. **Normalização (scraper/deconvolution)**: corrige fatores técnicos por célula preservando o sinal biológico; depende do *clustering* inicial.
4. **PCA (50 componentes)**: redução de dimensionalidade.
5. **Clusterização final (K-means, pós-PCA)**: produz o resultado que o *paper* de fato valida biologicamente.

O problema computacional central é que cada etapa tem um gargalo de natureza distinta. Algumas são limitadas pela leitura do disco (*I/O-bound*), outras pela transferência de dados entre CPU e GPU pelo barramento PCIe (*transfer-bound*) e outras pelo próprio cálculo (*compute-bound*). Tratar todas como “problemas de GPU” indistintamente levaria a decisões erradas de engenharia. O problema, então, não é apenas “acelerar”, mas decidir onde acelerar e com qual recurso.

3. Objetivos

Objetivo geral. Reimplementar o pipeline *mbkmeans* em arquitetura heterogênea (GPU + CPU *multicore*), superando o *baseline* em CPU *single-thread* do *paper* e analisando criticamente os ganhos.

Objetivos específicos:

- Estabelecer como *baseline* os tempos reportados por [1] e medir o *speedup* por etapa e *speedup* global da nossa implementação.
- Caracterizar, por medição, o gargalo de cada etapa e justificar a escolha GPU vs. CPU *multicore* em cada caso.
- Reformular as etapas mais custosas (normalização e PCA) de modo casado ao *hardware*, validando numericamente a corretude.
- Avaliar a qualidade do agrupamento com uma métrica (*silhouette score*) não usada pelo *paper*, discutindo o que ela revela sobre a clusterabilidade dos dados.

4. Metodologia

4.1. Princípio de arquitetura: a GPU só compensa quando o gargalo é cálculo

A GPU é poderosa no cálculo, mas o dado precisa viajar até ela ao longo do caminho SSD → RAM → VRAM. Se o tempo é gasto movendo dado (lendo do disco ou transferindo pela PCIe), acelerar o cálculo quase não muda o tempo total: a GPU fica ociosa esperando o dado (*data starvation*). Ela só rende quando o cálculo é pesado, regular e reutiliza dados já transferidos. Esse princípio orienta toda a alocação de trabalho entre GPU e CPU descrita a seguir, resumida na Tabela 2.

4.2. Pré-processamento (QC): GPU no Passe 1, CPU no Passe 2

O QC varre o HDF5 em dois passes. No Passe 1, as métricas de QC (contagem por célula, fração mitocondrial) são calculadas com somas prefixadas (*prefix sums/cumsum*) diretamente sobre os arranjos CSR, sem nunca densificar a matriz; o acesso é sequencial e regular, padrão em que a GPU é eficiente. No Passe 2, a matriz filtrada é montada num *buffer* pré-alocado, uma operação de movimentação de memória, não de cálculo. Esse passe roda em CPU *multicore* (*Numba* + *OpenMP*), paralelizando por célula: a escrita é irregular (cada célula tem número diferente de não-zeros, o que a GPU penaliza) e é embaraçosamente paralela, sem condição de corrida, pois as faixas de escrita não se sobrepõem, cenário ideal para CPU *multicore*.

4.3. Clustering inicial (*mini-batch K-means*) em GPU

O *clustering* inicial é implementado em GPU (*CuPy*): atribuição, somas por *cluster* e atualização dos centróides ocorrem na VRAM, com os centróides residentes na GPU. O algoritmo é *mini-batch* por necessidade de memória: a matriz $1,1\text{M} \times 13,9\text{k}$ esparsa não cabe na VRAM e é transmitida em lotes pela PCIe.

4.4. Normalização (*scrn*) reimplementada em GPU

O *paper* não descreve a normalização, ele a terceiriza para o *scrn* [2]. Reimplementamos fielmente o algoritmo de *deconvolution* do *scrn*: pré-escala por *library size*, ordenação das células, pseudo-célula de referência, *pools* circulares de tamanhos 21 a 101, fator de tamanho como mediana das razões *pool*/referência, deconvolução por sistema linear (LSQR), reescala entre *clusters* e centralização dos fatores em média 1. Optamos pelo *scrn* (e não por normalização simples por *library size*) porque a normalização por *library size* assume que toda célula tem a mesma composição de RNA; quando um tipo celular expressa mais RNA por razão biológica real, esse método trata o sinal como artefato e o apaga. O *scrn* compara cada célula com células semelhantes (*pooling*), respeitando diferenças biológicas e é justamente por exigir células homogêneas no *pool* que ele depende do clustering inicial. A GPU compensa aqui porque o cálculo por bloco é pesado e regular (divisões, medianas em janelas deslizantes e um *solve* linear) sobre um bloco denso e limitado (≤ 3000 células $\times$ $\sim 14k$ genes ≈ 170 MB) que cabe na VRAM, sem as milhares de idas e vindas pela PCIe que penalizam o *clustering*.

4.5. PCA: troca de algoritmo (IRLBA $\rightarrow$ covariância)

O *paper* usa o IRLBA [3] via BiocSingular para o PCA, que levou 96 horas. O IRLBA é um SVD truncado e iterativo que nunca forma $X^T X$; ele trabalha por produtos matriz-vetor e relê a matriz do disco a cada iteração. É um método genérico, e o preço da generalidade é I/O: dezenas de passadas pelo disco resultam nas 96 horas. O gargalo, aqui, é leitura repetida, não cálculo.

Como n (1,1M) $\gg p$ (13,9k genes), a matriz de covariância gene $\times$ gene é pequena ($\sim 1,5$ GB) e cabe na VRAM. Nosso método sai em exatamente duas passadas pelos dados: no Passe 1, acumula $S = X^T X$ por lote e as somas de coluna (média μ), e a identidade $M = X^T X - n \cdot \mu \mu^T$ centra os dados sem densificar X ; em seguida *eigh*(M) resolve os 50 maiores autovetores (*loadings*) diretamente na VRAM, sem releitura; no Passe 2, projeta os *scores* $= (X - \mu) \cdot V$ em *streaming*. O método é matematicamente exato: a autodecomposição de $(X - \mu)^T (X - \mu)$ fornece os mesmos componentes que o SVD de $X - \mu$. Reconhecemos como *trade-off* que formar $X^T X$ eleva ao quadrado o número de condicionamento; mitigamos isso acumulando em *float64* e retraindo apenas os 50 PCs mais robustos. O método é especializado (exige p pequeno o bastante para caber na VRAM), ao passo que o IRLBA funciona para p arbitrário.

4.6. Clusterização final: Lloyd completo, pós-PCA, com busca de K

Após o PCA, a matriz tem dimensão 1,1M $\times$ 50 (densa, ≈ 220 MB) e cabe inteira na VRAM. Não há mais motivo de memória para aproximar: usamos o algoritmo de Lloyd completo [4], em que cada iteração é essencialmente um único produto matricial $(n \times d) \cdot (d \times k)$, o padrão em que a GPU é imbatível. O próprio *paper* mostra que, para lotes ≥ 500 , o mini-batch é equivalente em acurácia ao *K-means* exato; logo, a troca por Lloyd completo remove a aproximação sem alterar as conclusões biológicas.

Nossa contribuição além do *paper* é uma busca sistemática sobre K (o *paper* fixa K=15): varremos K em diferentes faixas, com $n_{init}=3$ reinícios por K para evitar mínimos locais, e selecionamos K por evidência quantitativa: WCSS/n (método do cotovelo) e *silhouette score* amostrado [5]. A inclusão do *silhouette score* é deliberada: o *paper* avalia o agrupamento apenas por WCSS, métrica que decresce monotonicamente com K e não mede a separação/qualidade dos *clusters*.

4.7. Ambiente experimental

Os experimentos foram executados em um notebook equipado com GPU NVIDIA GeForce RTX 5070 Laptop (arquitetura *Blackwell*, 4608 núcleos CUDA, memória GDDR7), VRAM 8GB, 24 CPU-cores, 32GB RAM, 1 TB SSD NVme, `cupy-cuda13.x v14.1.0`, `numba v0.65.1`. Os tempos do *baseline* são os reportados por [1] (Tabela S5 do material suplementar), sobre o mesmo conjunto de ~1,3 milhão de células. Note-se que o *baseline* foi medido em hardware distinto (CPU single-thread); a comparação é, portanto, entre pipelines, e não entre unidades de hardware equivalentes.

5. Resultados

5.1. Desempenho: *speedup* por etapa e global

A Tabela 1 compara, etapa a etapa, os tempos do *baseline* [1] com os tempos medidos em nossa implementação. O ganho varia em ordens de magnitude conforme a etapa: de $\sim 1,7\times$ no pré-processamento (limitado por I/O) a $\sim 1080\times$ no PCA. Considerando as cinco etapas reimplementadas, o pipeline cai de $\sim 102,3$ horas para $\sim 26,47$ minutos, um *speedup* global de aproximadamente $232\times$.

Tabela 1. Tempos do *baseline* (*paper*, CPU single-thread) vs. nossa implementação (GPU + CPU multicore) e *speedup* por etapa.

Etapa	Baseline	Proposta	Speedup
Pré-processamento (QC)	8 min	4 min 40 s	$\sim 1,71\times$
Clustering inicial	8 min 30 s	7 min 14 s	$\sim 1,17\times$
Normalização (scrn)	5,18 h	8 min	$38,85\times$
PCA	96 h	5 min 20 s	$1080\times$
Clusterização final (busca de K)	~ 30 s	~ 70 s	$\sim 0,43\times$
Total (5 etapas)	$\sim 102,3$ h	$\sim 26,47$ min	$\sim 231,91\times$

* O *paper* executa o clustering pós-PCA em ~ 30 s, para K = 15. Nossa implementação varre 20 valores de K, com *Lloyd* completo e 3 reinícios, em 70 s. A comparação justa por *K-fit* também favorece a proposta.

5.2. Onde a GPU compensa: análise de gargalos

O resultado mais relevante do trabalho não é o número agregado, e sim a explicação de por que ele varia tanto. A Tabela 2 resume o gargalo de cada etapa e se a GPU compensa. Há um caso ilustrativo: o *clustering* inicial é implementado em GPU,

mas a GPU não traz ganho expressivo. Isso não é defeito, é um resultado medido. Cada uma das ~2000 iterações reenvia um novo lote pela PCIe; a GPU calcula as distâncias em microssegundos, mas a transferência custa milissegundos, e ela termina e fica ociosa (*data starvation*). Além disso, o acesso esparsa (CSR) exige indireção, o que quebra o acesso coalescido e desperdiça a largura de banda.

Tabela 2. Gargalo dominante de cada etapa e adequação da GPU.

Etapa	Gargalo	Natureza	GPU compensa?
Pré-proc. (QC)	SSD → RAM (I/O)	I/O-bound	Pouco
Clustering inicial	PCIe (repetido)	Transfer-bound + esparsa	Não
Normalização	Cálculo por bloco	Compute-bound	Sim
PCA	(algorítmico)	Compute-bound, denso	Sim, muito
Clusterização final	Cálculo denso	Compute-bound, denso	Sim, decisivo

Destacamos ainda que a maior parte do *speedup* do PCA (96 h → 5min20s) é algorítmica, não da GPU: reconhecer que $n \gg p$ e trocar o SVD iterativo *out-of-core* por uma covariância em duas passadas esmagaria o IRLBA mesmo em CPU. A GPU atua como multiplicador (acelera o *eigh* e os produtos matriciais), e não como fonte primária do ganho.

5.3. Validação numérica

Para garantir que a aceleração não comprometeu a corretude, validamos cada reformulação. A normalização foi conferida em *NumPy* contra fatores de tamanho conhecidos, obtendo correlação de 0,99 (um *cluster*) e 0,997 (*multi-cluster*), com erro de ~2,6–6% (vs. ~15–23% da normalização simples por *library size*). O PCA foi validado contra o PCA do *scikit-learn*, com correlação 1,000000 nos 50 componentes. Um validador de integridade percorreu o pipeline ponta a ponta (bruto → filtrado → clusterizado → normalizado → PCA) em *streaming*, confirmando consistência (1.148.558 células em todas as etapas, 0 barcodes duplicados, 13.897 genes consistentes).

5.4. Qualidade do agrupamento: WCSS vs. *silhouette*

As Figura 1 e Figura 2 apresentam, em função de K , a WCSS/ n (método do cotovelo) e o coeficiente de *silhouette*. A WCSS decresce de forma suave, sem um cotovelo forte e bem definido. O *silhouette score*, por sua vez, permanece uniformemente baixo (≈ 0) para praticamente todos os valores de K , com um pico isolado em $K=2$ ($\sim 0,20$), para a figura 1, e em $K=5$ ($\sim 0,03$), para a figura 2, seguido de degradação rápida. Esse padrão se manteve consistente entre faixas e rodadas

independentes de teste, indício de que é sinal real, e não ruído de execução. Por convenção, *silhouette_score* < 0,25 indica estrutura de *cluster* fraca ou ausente.

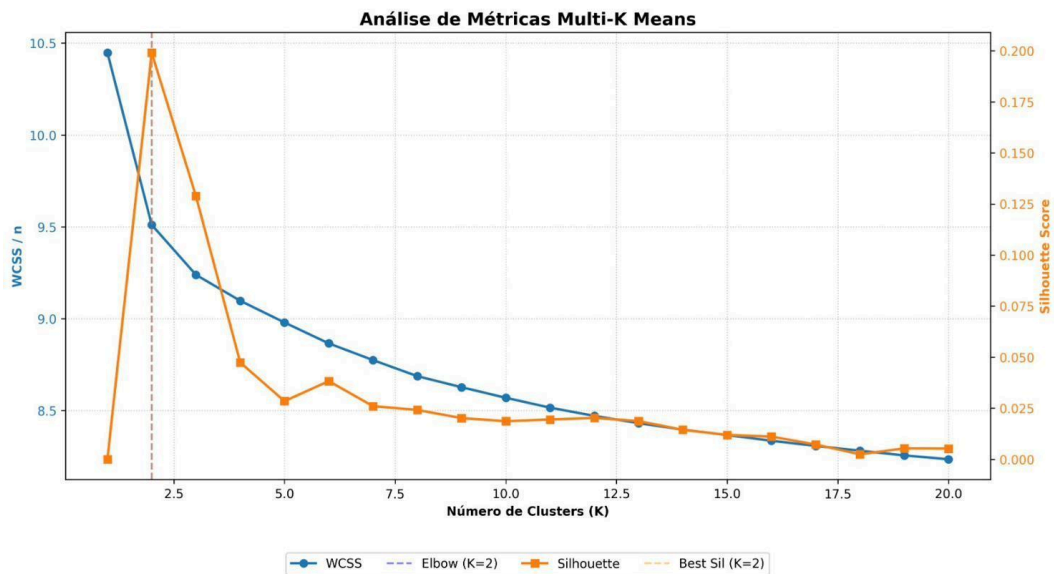


Figura 1. Análise *multi-K*: WCSS/n e *silhouette* para K de 1 a 20. O *silhouette* tem pico isolado em K=2 (~0,20) e colapsa para ≈ 0 nos demais K, indicando ausência de estrutura globular bem separada.

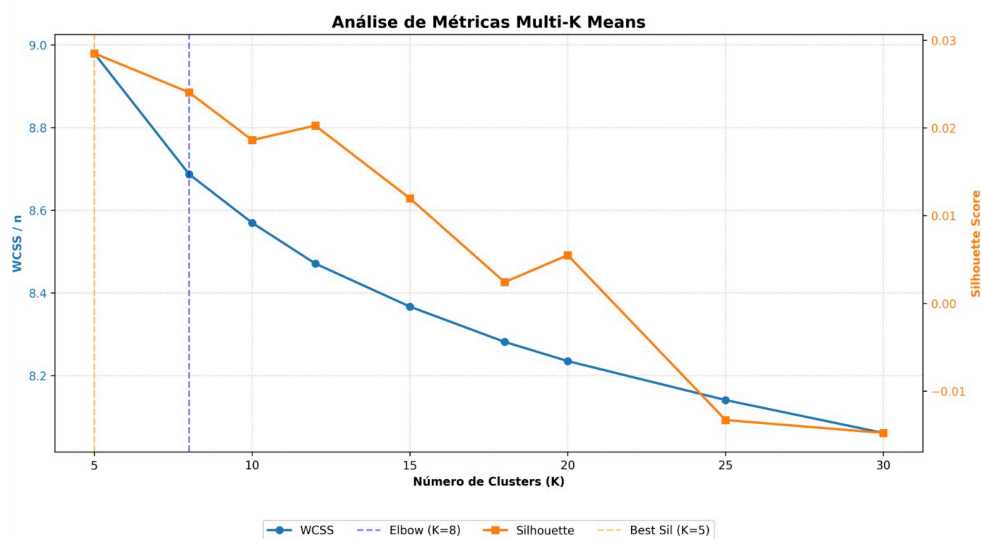


Figura 2. Análise *multi-K*: WCSS/n e *silhouette* para K de 5 a 30. O *silhouette* tem pico isolado em K=5 (~0,03) e colapsa para ≈ 0 nos demais K.

5.5. Visualização PCA e UMAP e a hipótese de não-clusterabilidade

Diante do *silhouette* consistentemente baixo, projetamos os *clusters* no espaço dos dois primeiros componentes principais para inspeção visual (Figura 3). As projeções mostram forte sobreposição entre os *clusters*, sem fronteiras nítidas, e o aumento de K não produz separação, apenas fatia uma nuvem contínua em mais regiões arbitrárias.

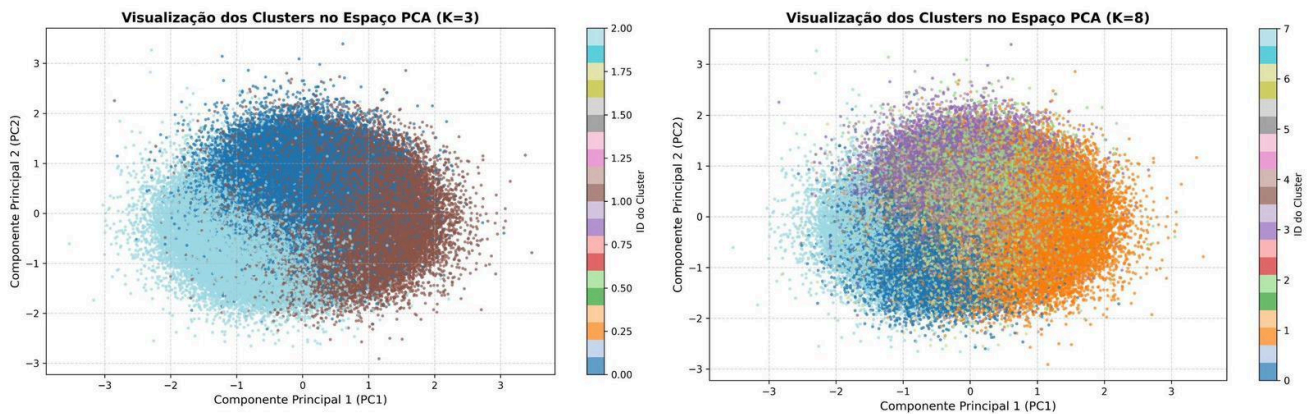


Figura 3. Projeção dos *clusters* no espaço PCA ($PC1 \times PC2$) para $K=3$ (esquerda) e $K=8$ (direita). Aumentar K não cria fronteiras: os pontos formam uma nuvem contínua, coerente com o *silhouette* baixo.

Cabe uma ressalva metodológica: a projeção PCA em duas dimensões não prova, por si só, que os dados originais não sejam agrupáveis. A usamos como evidência de apoio, combinada à evidência primária (*silhouette score* baixo, consistente entre rodadas), para levantar a hipótese de que os dados não são bem separáveis por hipersferas, que é a forma de *cluster* assumida pelo *K-means* com distância euclidiana. Como o PCA preserva a variância dos dados originais ao reduzir a dimensionalidade, é razoável conjecturar que a dificuldade de separação observada reflete a estrutura subjacente dos dados, e não um artefato da projeção. Uma hipótese biológica plausível é que dados de neurônios formam um contínuo de diferenciação/maturação, em vez de *clusters* discretos e bem delimitados; nesse regime, qualquer K produz *silhouette* baixo, pois não há fronteiras nítidas a encontrar. Pode ser observado, na Figura 4, que para $K=15$, a projeção UMAP dos dados clusterizados, reforça a distribuição não-esférica dos dados biológicos. Sendo plausível pensar que os dados de neurônios de camundongo formam um contínuo de diferenciação que poderia ser melhor capturado por sobreposições de cônicas ou outras estruturas de dados.

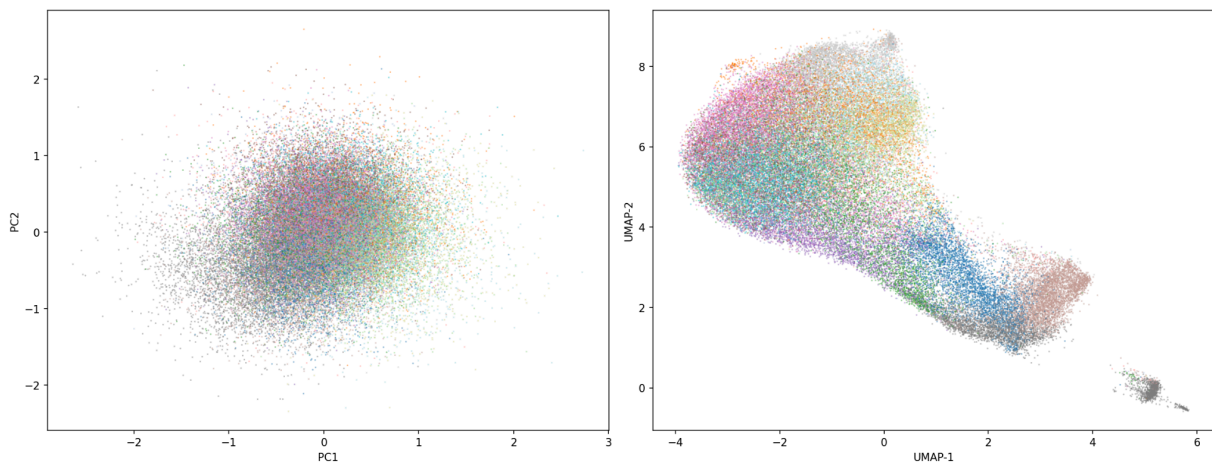


Figura 4. Projeção dos *clusters* no espaço PCA ($PC1 \times PC2$): $K=15$ (esq.) e UMAP(UMAP-1 x UMAP-2): $K=15$ (dir). Uma nuvem contínua, sem fronteiras nítidas.

Esse achado motiva, como trabalho futuro, o uso de modelos que admitem *clusters* de forma elipsoidal, como misturas de gaussianas (GMM), ou de métodos baseados em grafo, como *Louvain* e *Leiden* sobre KNN, que a literatura moderna de *scRNA-seq* frequentemente prefere ao *K-means* para esse tipo de estrutura; de fato, são exatamente as abordagens com que [1] comparam o *mbkmeans*. Importante: isso não invalida nossa implementação, é uma limitação conhecida do método (*K-means*/euclidiana) frente ao regime do dado, e o *paper* de referência não a avalia, pois sua única métrica de agrupamento (WCSS) não mede qualidade de separação.

6. Conclusão

Reimplementamos o *pipeline mbkmeans* de *scRNA-seq* em arquitetura heterogênea (GPU + CPU *multicore*), reduzindo o tempo das cinco etapas reimplementadas de ~102 horas para ~26 minutos (*speedup* de ~232×). Mais importante que o número agregado, mostramos por medição por que o ganho varia tanto entre etapas: o gargalo de cada uma é diferente (I/O, PCIe ou cálculo), e a GPU só compensa quando o gargalo é cálculo pesado e regular sobre dados que cabem na VRAM. Documentamos, inclusive, um caso em que rodar na GPU não compensa (o *clustering* inicial), o que reforça a tese central. Reconhecemos também que o maior ganho isolado (PCA) é majoritariamente algorítmico, com a GPU atuando como multiplicador.

Por fim, ao introduzir uma métrica de qualidade (*silhouette score*) ausente no *paper*, obtivemos evidência de que os dados não são bem separados por *K-means*, levantando a hipótese, a ser investigada com GMM ou métodos de grafo, de que sua estrutura é mais contínua do que discreta. Como trabalho futuro, além de avaliar esses métodos alternativos, pretendemos reportar o tempo médio por *K-fit* da versão final e detalhar o ambiente experimental para uma comparação de desempenho plenamente reprodutível.

Disponibilidade do código

O código (*notebooks* e fontes) está disponível em <https://github.com/IgorGMMazo/cuda-multi-k-means-scRNA>.

Referências

- [1] Hicks, S. C., Liu, R., Ni, Y., Purdom, E., and Risso, D. (2021). *mbkmeans*: Fast clustering for single cell data using mini-batch k-means. *PLOS Computational Biology*, 17(1):e1008625.
- [2] Lun, A. T. L., Bach, K., and Marioni, J. C. (2016). Pooling across cells to normalize single-cell RNA sequencing data with many zero counts. *Genome Biology*, 17:75.
- [3] Baglama, J. and Reichel, L. (2005). Augmented implicitly restarted Lanczos bidiagonalization methods. *SIAM Journal on Scientific Computing*, 27(1):19–42.
- [4] Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.

- [5] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.